COMP3153/9153

Algorithmic Verification

Lecture 11: Abstract Interpretation and Static Analysis

Static Analysis

Static analysis is the automatic analysis of code without executing it. It has a variety of applications from security to performance optimisations.

Static Analysis

Static analysis is the automatic analysis of code without executing it. It has a variety of applications from security to performance optimisations.

Rice's Theorem

Any non-trivial property about the language recognized by a Turing machine is undecidable.

Static Analysis

Static analysis is the automatic analysis of code without executing it. It has a variety of applications from security to performance optimisations.

Rice's Theorem

Any non-trivial property about the language recognized by a Turing machine is undecidable.

Our trick: Abstract away from the non-computable or intractable bits by approximating.

Static Analysis

Static analysis is the automatic analysis of code without executing it. It has a variety of applications from security to performance optimisations.

Rice's Theorem

Any non-trivial property about the language recognized by a Turing machine is undecidable.

Our trick: Abstract away from the non-computable or intractable bits by approximating.

Caution

If we overapproximate, we may produce a lot of *false positives* (spurious errors), but our analysis will be *sound*.

Static Analysis

Static analysis is the automatic analysis of code without executing it. It has a variety of applications from security to performance optimisations.

Rice's Theorem

Any non-trivial property about the language recognized by a Turing machine is undecidable.

Our trick: Abstract away from the non-computable or intractable bits by approximating.

Caution

If we overapproximate, we may produce a lot of *false positives* (spurious errors), but our analysis will be *sound*. If we underapproximate, we may report that the program is fine when it isn't (*false negatives*), but our analysis will be *complete*.

Static Analysis

Static analysis is the automatic analysis of code without executing it. It has a variety of applications from security to performance optimisations.

Rice's Theorem

Any non-trivial property about the language recognized by a Turing machine is undecidable.

Our trick: Abstract away from the non-computable or intractable bits by approximating.

Caution

If we overapproximate, we may produce a lot of *false positives* (spurious errors), but our analysis will be *sound*. If we underapproximate, we may report that the program is fine when it isn't (*false negatives*), but our analysis will be *complete*.

Most tools aren't sound nor complete, because they're a mixture.

Abstract Interpretation

A very common use-case for fixed point computations is in *abstract interpretation*, a type of static analysis.

Key Idea

1. Replace concrete variables with approximate abstractions in an *abstract domain*, which is a lattice.

Abstract Interpretation

A very common use-case for fixed point computations is in *abstract interpretation*, a type of static analysis.

Key Idea

1. Replace concrete variables with approximate abstractions in an *abstract domain*, which is a lattice.
2. Approximate the program's semantics using monotonic functions defined over that domain.

Abstract Interpretation

A very common use-case for fixed point computations is in *abstract interpretation*, a type of static analysis.

Key Idea

1. Replace concrete variables with approximate abstractions in an *abstract domain*, which is a lattice.
2. Approximate the program's semantics using monotonic functions defined over that domain.
3. Compute the least fixed point of these functions.

Abstract Interpretation

A very common use-case for fixed point computations is in *abstract interpretation*, a type of static analysis.

Key Idea

1. Replace concrete variables with approximate abstractions in an *abstract domain*, which is a lattice.
2. Approximate the program's semantics using monotonic functions defined over that domain.
3. Compute the least fixed point of these functions.

We have seen this before. Predicate abstraction is an example of abstract interpretation.

The WHILE Language

To make things easier, we will define a simple imperative programming language as follows:

$$
\begin{array}{rcl}
\mathcal{A} & ::= & \langle \textit{arithmetic expressions} \rangle \\
\mathcal{B} & ::= & \langle \textit{boolean expressions} \rangle \\
\mathcal{S} & ::= & [x := \mathcal{A}]^{\ell} \mid [\textbf{skip}]^{\ell} \mid \mathcal{S}_1 ; \mathcal{S}_2 \\
 & \mid & \textbf{if } [\mathcal{B}]^{\ell} \textbf{ then } \mathcal{S} \textbf{ else } \mathcal{S} \\
 & \mid & \textbf{while } [\mathcal{B}]^{\ell} \textbf{ do } \mathcal{S}
\end{array}
$$

Note that we *label* all statements and conditions (usually with a number). These labelled terms correspond to nodes on the control flow graph.

Examples

$$
\begin{array}{l}
\textbf{if } [x > 0]^1 \textbf{ then} \\
\quad [x := 2x + 1]^2 \\
\textbf{else} \\
\quad [x := 1 - 4x]^3 \\
[x := 8 \div x]^4
\end{array}
$$

Examples

if $[x > 0]^1$ **then**
    $[x := 2x + 1]^2$
else
    $[x := 1 - 4x]^3$
$[x := 8 \div x]^4$

$$\begin{tikzcd}
 & {[x > 0]^1} \arrow[dl] \arrow[dr] & \\
{[x := 2x + 1]^2} \arrow[dr] & & {[x := 1 - 4x]^3} \arrow[dl] \\
 & {[x := 8 \div x]^4} &
\end{tikzcd}$$

Examples

if $[x > 0]^1$ **then**
    $[x := 2x + 1]^2$
else
    $[x := 1 - 4x]^3$
$[x := 8 \div x]^4$

$$
\begin{tikzcd}
 & {[x > 0]^1} \arrow[dl] \arrow[dr] & \\
{[x := 2x + 1]^2} \arrow[dr] & & {[x := 1 - 4x]^3} \arrow[dl] \\
 & {[x := 8 \div x]^4} &
\end{tikzcd}
$$

What abstract domain should we use to detect divide by zero?

Intervals

Let's define intervals n, m as either:

- $\emptyset$, the empty interval, or
- $[n, m]$ where $n, m \in \mathbb{Z} \cup \{+\infty, -\infty\}$.

Intervals

Let's define intervals n, m as either:

- $\emptyset$, the empty interval, or
- $[n, m]$ where $n, m \in \mathbb{Z} \cup \{+\infty, -\infty\}$.

We can define interval intersection $n \cap m$ as the interval where n and m overlap.

Intervals

Let's define intervals n, m as either:

- $\emptyset$, the empty interval, or
- $[n, m]$ where $n, m \in \mathbb{Z} \cup \{+\infty, -\infty\}$.

We can define interval intersection $n \cap m$ as the interval where n and m overlap.
Likewise, define union $n \cup m$ as the smallest interval containing both n and m.

Intervals

Let's define intervals n, m as either:

- $\emptyset$, the empty interval, or
- $[n, m]$ where $n, m \in \mathbb{Z} \cup \{+\infty, -\infty\}$.

We can define interval intersection $n \cap m$ as the interval where n and m overlap.
Likewise, define union $n \cup m$ as the smallest interval containing both n and m.

Observation

Define $\inf S = \bigcap S$ and $\sup S = \bigcup S$, then intervals form a lattice: $\bot = \emptyset$ and $\top = [-\infty, +\infty]$. The ordering $\preceq$ here is interval inclusion.

Intervals

Let's define intervals n, m as either:

- $\emptyset$, the empty interval, or
- $[n, m]$ where $n, m \in \mathbb{Z} \cup \{+\infty, -\infty\}$.

We can define interval intersection $n \cap m$ as the interval where n and m overlap.
Likewise, define union $n \cup m$ as the smallest interval containing both n and m.

Observation

Define $\inf S = \bigcap S$ and $\sup S = \bigcup S$, then intervals form a lattice: $\bot = \emptyset$ and $\top = [-\infty, +\infty]$. The ordering $\preceq$ here is interval inclusion.

We can "lift" arithmetic operators to the interval level, where they apply to both bounds. Similarly define e.g. $\hat{3} = [3, 3]$.

Equation Systems

We define a series of *entry* interval equations x_i, and a series of *exit* equations x_i' describing the possible values of x before and after the statement i.

$$x_1 \quad =$$

$$\begin{tikzcd}
 & {[x > 0]^1} \arrow[dl] \arrow[dr] & \\
{[x := 2x + 1]^2} \arrow[dr] & & {[x := 1 - 4x]^3} \arrow[dl] \\
 & {[x := 8 \div x]^4} &
\end{tikzcd}$$

Equation Systems

We define a series of *entry* interval equations x_i, and a series of *exit* equations x_i' describing the possible values of x before and after the statement i.

$$
\begin{aligned}
x_1 &= [-\infty, +\infty] \\
x_1' &=
\end{aligned}
$$

$$
\begin{tikzcd}
& {[x > 0]^1} \arrow[dl] \arrow[dr] & \\
{[x := 2x + 1]^2} \arrow[dr] & & {[x := 1 - 4x]^3} \arrow[dl] \\
& {[x := 8 \div x]^4} &
\end{tikzcd}
$$

Equation Systems

We define a series of *entry* interval equations x_i, and a series of *exit* equations x_i' describing the possible values of x before and after the statement i.

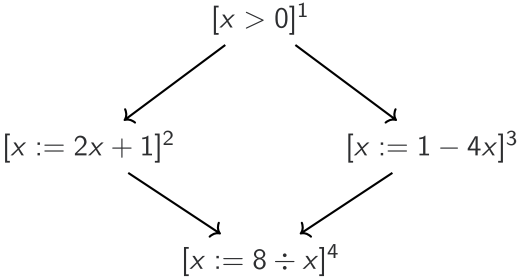
$$
\begin{aligned}
x_1 &= [-\infty, +\infty] \\
x_1' &= x_1 \\
x_2 &=
\end{aligned}
$$

$$
\begin{tikzcd}
& {[x > 0]^1} \arrow[dl] \arrow[dr] & \\
{[x := 2x + 1]^2} \arrow[dr] & & {[x := 1 - 4x]^3} \arrow[dl] \\
& {[x := 8 \div x]^4} &
\end{tikzcd}
$$

Equation Systems

We define a series of *entry* interval equations x_i, and a series of *exit* equations x'_i describing the possible values of x before and after the statement i.

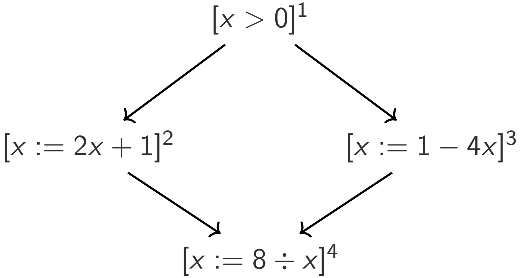
$$
\begin{aligned}
x_1 &= [-\infty, +\infty] \\
x'_1 &= x_1 \\
x_2 &= x'_1 \cap [1, +\infty] \\
x'_2 &=
\end{aligned}
$$



Equation Systems

We define a series of *entry* interval equations x_i, and a series of *exit* equations x'_i describing the possible values of x before and after the statement i.

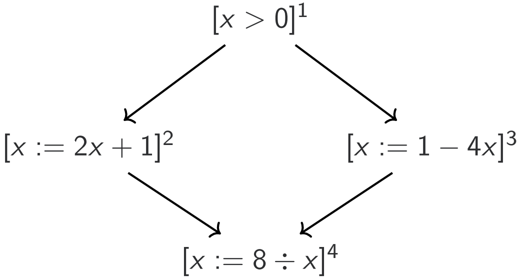
$$\begin{aligned}
x_1 &= [-\infty, +\infty] \\
x'_1 &= x_1 \\
x_2 &= x'_1 \cap [1, +\infty] \\
x'_2 &= \hat{2} \times x_2 + \hat{1} \\
x_3 &=
\end{aligned}$$



Equation Systems

We define a series of *entry* interval equations x_i, and a series of *exit* equations x'_i describing the possible values of x before and after the statement i.

$$\begin{aligned}
x_1 &= [-\infty, +\infty] \\
x'_1 &= x_1 \\
x_2 &= x'_1 \cap [1, +\infty] \\
x'_2 &= \hat{2} \times x_2 + \hat{1} \\
x_3 &= x'_1 \cap [-\infty, 0] \\
x'_3 &=
\end{aligned}$$



Equation Systems

We define a series of *entry* interval equations x_i, and a series of *exit* equations x_i' describing the possible values of x before and after the statement i.

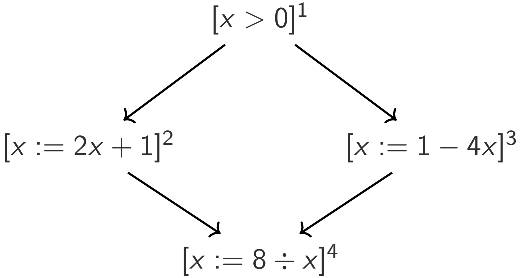
$$
\begin{aligned}
x_1 &= [-\infty, +\infty] \\
x_1' &= x_1 \\
x_2 &= x_1' \cap [1, +\infty] \\
x_2' &= \hat{2} \times x_2 + \hat{1} \\
x_3 &= x_1' \cap [-\infty, 0] \\
x_3' &= \hat{1} - \hat{4} \times x_3 \\
x_4 &=
\end{aligned}
$$

$$
\begin{tikzcd}
 & {[x > 0]^1} \arrow[dl] \arrow[dr] & \\
{[x := 2x + 1]^2} \arrow[dr] & & {[x := 1 - 4x]^3} \arrow[dl] \\
 & {[x := 8 \div x]^4} &
\end{tikzcd}
$$

Equation Systems

We define a series of *entry* interval equations x_i, and a series of *exit* equations x_i' describing the possible values of x before and after the statement i.

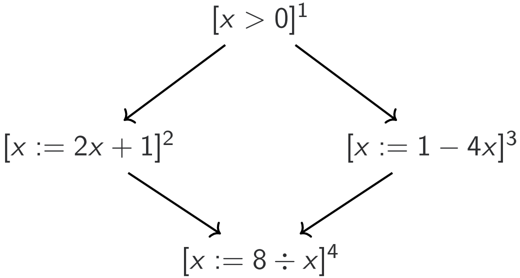
$$
\begin{aligned}
x_1 &= [-\infty, +\infty] \\
x_1' &= x_1 \\
x_2 &= x_1' \cap [1, +\infty] \\
x_2' &= \hat{2} \times x_2 + \hat{1} \\
x_3 &= x_1' \cap [-\infty, 0] \\
x_3' &= \hat{1} - \hat{4} \times x_3 \\
x_4 &= x_2' \cup x_3' \\
x_4' &=
\end{aligned}
$$



Equation Systems

We define a series of *entry* interval equations x_i, and a series of *exit* equations x'_i describing the possible values of x before and after the statement i.

$$
\begin{aligned}
x_1 &= [-\infty, +\infty] \\
x'_1 &= x_1 \\
x_2 &= x'_1 \cap [1, +\infty] \\
x'_2 &= \hat{2} \times x_2 + \hat{1} \\
x_3 &= x'_1 \cap [-\infty, 0] \\
x'_3 &= \hat{1} - \hat{4} \times x_3 \\
x_4 &= x'_2 \cup x'_3 \\
x'_4 &= \hat{8} \div x_4
\end{aligned}
$$



Equation Systems

We define a series of *entry* interval equations x_i, and a series of *exit* equations x'_i describing the possible values of x before and after the statement i.

$$\begin{aligned}
x_1 &= [-\infty, +\infty] \\
x'_1 &= x_1 \\
x_2 &= x'_1 \cap [1, +\infty] \\
x'_2 &= \hat{2} \times x_2 + \hat{1} \\
x_3 &= x'_1 \cap [-\infty, 0] \\
x'_3 &= \hat{1} - \hat{4} \times x_3 \\
x_4 &= x'_2 \cup x'_3 \\
x'_4 &= \hat{8} \div x_4
\end{aligned}$$

$$\begin{tikzcd}
& {[x > 0]^1} \arrow[dl] \arrow[dr] & \\
{[x := 2x + 1]^2} \arrow[dr] & & {[x := 1 - 4x]^3} \arrow[dl] \\
& {[x := 8 \div x]^4} &
\end{tikzcd}$$

Observe that all operations used are monotone. How can we compute what the intervals are? Iterate to the least fixed point !

Least Fixed Points for Equation Systems

We start initialising all equations to $\perp$, and then iterate until the results stop changing. We can choose the equations in any (fair) order, and we will always reach a fixed point eventually. Some ways are faster than others.

$$
\begin{array}{lclcl}
x_1 & = & \emptyset & = & [-\infty, +\infty] \\
x_1' & = & \emptyset & = & x_1 \\
x_2 & = & \emptyset & = & x_1' \cap [1, +\infty] \\
x_2' & = & \emptyset & = & \hat{2} \times x_2 + \hat{1} \\
x_3 & = & \emptyset & = & x_1' \cap [-\infty, 0] \\
x_3' & = & \emptyset & = & \hat{1} - \hat{4} \times x_3 \\
x_4 & = & \emptyset & = & x_2' \cup x_3' \\
x_4' & = & \emptyset & = & \hat{8} \div x_4
\end{array}
$$

Least Fixed Points for Equation Systems

We start initialising all equations to $\bot$, and then iterate until the results stop changing. We can choose the equations in any (fair) order, and we will always reach a fixed point eventually. Some ways are faster than others.

$$
\begin{array}{lcl}
x_1 & = & \emptyset = [-\infty, +\infty] & = & [-\infty, +\infty] \\
x_1' & = & \emptyset & = & x_1 \\
x_2 & = & \emptyset & = & x_1' \cap [1, +\infty] \\
x_2' & = & \emptyset & = & \hat{2} \times x_2 + \hat{1} \\
x_3 & = & \emptyset & = & x_1' \cap [-\infty, 0] \\
x_3' & = & \emptyset & = & \hat{1} - \hat{4} \times x_3 \\
x_4 & = & \emptyset & = & x_2' \cup x_3' \\
x_4' & = & \emptyset & = & \hat{8} \div x_4
\end{array}
$$

Least Fixed Points for Equation Systems

We start initialising all equations to $\perp$, and then iterate until the results stop changing. We can choose the equations in any (fair) order, and we will always reach a fixed point eventually. Some ways are faster than others.

$$
\begin{array}{lcllcl}
x_1 & = & \emptyset = [-\infty, +\infty] & = & [-\infty, +\infty] \\
x_1' & = & \emptyset = [-\infty, +\infty] & = & x_1 \\
x_2 & = & \emptyset & = & x_1' \cap [1, +\infty] \\
x_2' & = & \emptyset & = & \hat{2} \times x_2 + \hat{1} \\
x_3 & = & \emptyset & = & x_1' \cap [-\infty, 0] \\
x_3' & = & \emptyset & = & \hat{1} - \hat{4} \times x_3 \\
x_4 & = & \emptyset & = & x_2' \cup x_3' \\
x_4' & = & \emptyset & = & \hat{8} \div x_4
\end{array}
$$

Least Fixed Points for Equation Systems

We start initialising all equations to $\perp$, and then iterate until the results stop changing. We can choose the equations in any (fair) order, and we will always reach a fixed point eventually. Some ways are faster than others.

$$\begin{array}{lclcl}
x_1 & = & \emptyset = [-\infty, +\infty] & = & [-\infty, +\infty] \\
x_1' & = & \emptyset = [-\infty, +\infty] & = & x_1 \\
x_2 & = & \emptyset = [1, +\infty] & = & x_1' \cap [1, +\infty] \\
x_2' & = & \emptyset & = & \hat{2} \times x_2 + \hat{1} \\
x_3 & = & \emptyset & = & x_1' \cap [-\infty, 0] \\
x_3' & = & \emptyset & = & \hat{1} - \hat{4} \times x_3 \\
x_4 & = & \emptyset & = & x_2' \cup x_3' \\
x_4' & = & \emptyset & = & \hat{8} \div x_4
\end{array}$$

Least Fixed Points for Equation Systems

We start initialising all equations to $\perp$, and then iterate until the results stop changing. We can choose the equations in any (fair) order, and we will always reach a fixed point eventually. Some ways are faster than others.

$$
\begin{array}{lcllcl}
x_1 & = & \emptyset = [-\infty, +\infty] & = & [-\infty, +\infty] \\
x_1' & = & \emptyset = [-\infty, +\infty] & = & x_1 \\
x_2 & = & \emptyset = [1, +\infty] & = & x_1' \cap [1, +\infty] \\
x_2' & = & \emptyset = [3, +\infty] & = & \hat{2} \times x_2 + \hat{1} \\
x_3 & = & \emptyset & = & x_1' \cap [-\infty, 0] \\
x_3' & = & \emptyset & = & \hat{1} - \hat{4} \times x_3 \\
x_4 & = & \emptyset & = & x_2' \cup x_3' \\
x_4' & = & \emptyset & = & \hat{8} \div x_4
\end{array}
$$

Least Fixed Points for Equation Systems

We start initialising all equations to $\bot$, and then iterate until the results stop changing. We can choose the equations in any (fair) order, and we will always reach a fixed point eventually. Some ways are faster than others.

$$
\begin{array}{lcllcl}
x_1 & = & \emptyset = [-\infty, +\infty] & = & [-\infty, +\infty] \\
x_1' & = & \emptyset = [-\infty, +\infty] & = & x_1 \\
x_2 & = & \emptyset = [1, +\infty] & = & x_1' \cap [1, +\infty] \\
x_2' & = & \emptyset = [3, +\infty] & = & \hat{2} \times x_2 + \hat{1} \\
x_3 & = & \emptyset = [-\infty, 0] & = & x_1' \cap [-\infty, 0] \\
x_3' & = & \emptyset & = & \hat{1} - \hat{4} \times x_3 \\
x_4 & = & \emptyset & = & x_2' \cup x_3' \\
x_4' & = & \emptyset & = & \hat{8} \div x_4
\end{array}
$$

Least Fixed Points for Equation Systems

We start initialising all equations to $\perp$, and then iterate until the results stop changing. We can choose the equations in any (fair) order, and we will always reach a fixed point eventually. Some ways are faster than others.

$$
\begin{array}{lcllcl}
x_1 & = & \emptyset = [-\infty, +\infty] & = & [-\infty, +\infty] \\
x_1' & = & \emptyset = [-\infty, +\infty] & = & x_1 \\
x_2 & = & \emptyset = [1, +\infty] & = & x_1' \cap [1, +\infty] \\
x_2' & = & \emptyset = [3, +\infty] & = & \hat{2} \times x_2 + \hat{1} \\
x_3 & = & \emptyset = [-\infty, 0] & = & x_1' \cap [-\infty, 0] \\
x_3' & = & \emptyset = [1, +\infty] & = & \hat{1} - \hat{4} \times x_3 \\
x_4 & = & \emptyset & = & x_2' \cup x_3' \\
x_4' & = & \emptyset & = & \hat{8} \div x_4
\end{array}
$$

Least Fixed Points for Equation Systems

We start initialising all equations to $\perp$, and then iterate until the results stop changing. We can choose the equations in any (fair) order, and we will always reach a fixed point eventually. Some ways are faster than others.

$$
\begin{array}{lcllcl}
x_1 & = & \emptyset = [-\infty, +\infty] & = & [-\infty, +\infty] \\
x_1' & = & \emptyset = [-\infty, +\infty] & = & x_1 \\
x_2 & = & \emptyset = [1, +\infty] & = & x_1' \cap [1, +\infty] \\
x_2' & = & \emptyset = [3, +\infty] & = & \hat{2} \times x_2 + \hat{1} \\
x_3 & = & \emptyset = [-\infty, 0] & = & x_1' \cap [-\infty, 0] \\
x_3' & = & \emptyset = [1, +\infty] & = & \hat{1} - \hat{4} \times x_3 \\
x_4 & = & \emptyset = [1, +\infty] & = & x_2' \cup x_3' \\
x_4' & = & \emptyset & = & \hat{8} \div x_4
\end{array}
$$

Least Fixed Points for Equation Systems

We start initialising all equations to $\bot$, and then iterate until the results stop changing. We can choose the equations in any (fair) order, and we will always reach a fixed point eventually. Some ways are faster than others.

$$
\begin{array}{lclcl}
x_1 & = & \emptyset = [-\infty, +\infty] & = & [-\infty, +\infty] \\
x_1' & = & \emptyset = [-\infty, +\infty] & = & x_1 \\
x_2 & = & \emptyset = [1, +\infty] & = & x_1' \cap [1, +\infty] \\
x_2' & = & \emptyset = [3, +\infty] & = & \hat{2} \times x_2 + \hat{1} \\
x_3 & = & \emptyset = [-\infty, 0] & = & x_1' \cap [-\infty, 0] \\
x_3' & = & \emptyset = [1, +\infty] & = & \hat{1} - \hat{4} \times x_3 \\
x_4 & = & \emptyset = [1, +\infty] & = & x_2' \cup x_3' \\
x_4' & = & \emptyset = [0, 8] & = & \hat{8} \div x_4
\end{array}
$$

Least Fixed Points for Equation Systems

We start initialising all equations to $\bot$, and then iterate until the results stop changing. We can choose the equations in any (fair) order, and we will always reach a fixed point eventually. Some ways are faster than others.

$$
\begin{array}{lcllcl}
x_1 & = & \emptyset = [-\infty, +\infty] & = & [-\infty, +\infty] \\
x_1' & = & \emptyset = [-\infty, +\infty] & = & x_1 \\
x_2 & = & \emptyset = [1, +\infty] & = & x_1' \cap [1, +\infty] \\
x_2' & = & \emptyset = [3, +\infty] & = & \hat{2} \times x_2 + \hat{1} \\
x_3 & = & \emptyset = [-\infty, 0] & = & x_1' \cap [-\infty, 0] \\
x_3' & = & \emptyset = [1, +\infty] & = & \hat{1} - \hat{4} \times x_3 \\
x_4 & = & \emptyset = [1, +\infty] & = & x_2' \cup x_3' \\
x_4' & = & \emptyset = [0, 8] & = & \hat{8} \div x_4
\end{array}
$$

Seeing as $0 \notin x_4$, we know divide by zero is impossible.

Slow Convergence

Because the previous example had no loops, all equations converged after one step. Compare to this example:

$$
\begin{array}{l}
[n := 1]^1 \\
\textbf{while } [n < 1000]^2 \textbf{ do} \\
\quad [n := n + 1]^3 \\
[\textbf{skip}]^4
\end{array}
$$

Slightly simplified equations for presentation:

$$
\begin{array}{lclcl}
n'_1 & = & \emptyset & = & [1, 1] \\
n_2 & = & \emptyset & = & n'_1 \cup n'_3 \\
n_3 & = & \emptyset & = & n_2 \cap [-\infty, 999] \\
n'_3 & = & \emptyset & = & n_3 + 1 \\
n_4 & = & \emptyset & = & n'_3 \cap [1000, +\infty]
\end{array}
$$

Slow Convergence

Because the previous example had no loops, all equations converged after one step. Compare to this example:

$$
\begin{array}{l}
[n := 1]^1 \\
\textbf{while } [n < 1000]^2 \textbf{ do} \\
\quad [n := n + 1]^3 \\
[\textbf{skip}]^4
\end{array}
$$

Slightly simplified equations for presentation:

$$
\begin{array}{lcl}
n'_1 & = & \emptyset = [1,1] & = & [1,1] \\
n_2 & = & \emptyset & = & n'_1 \cup n'_3 \\
n_3 & = & \emptyset & = & n_2 \cap [-\infty, 999] \\
n'_3 & = & \emptyset & = & n_3 + 1 \\
n_4 & = & \emptyset & = & n'_3 \cap [1000, +\infty]
\end{array}
$$

Slow Convergence

Because the previous example had no loops, all equations converged after one step. Compare to this example:

$$
\begin{array}{l}
[n := 1]^1 \\
\textbf{while } [n < 1000]^2 \textbf{ do} \\
\quad [n := n + 1]^3 \\
[\textbf{skip}]^4
\end{array}
$$

Slightly simplified equations for presentation:

$$
\begin{array}{lcl@{\qquad\qquad}cl}
n'_1 & = & \emptyset = [1,1] & = & [1,1] \\
n_2 & = & \emptyset = [1,1] & = & n'_1 \cup n'_3 \\
n_3 & = & \emptyset & = & n_2 \cap [-\infty, 999] \\
n'_3 & = & \emptyset & = & n_3 + 1 \\
n_4 & = & \emptyset & = & n'_3 \cap [1000, +\infty]
\end{array}
$$

Slow Convergence

Because the previous example had no loops, all equations converged after one step. Compare to this example:

$$
\begin{array}{l}
[n := 1]^1 \\
\textbf{while } [n < 1000]^2 \textbf{ do} \\
\quad [n := n + 1]^3 \\
[\textbf{skip}]^4
\end{array}
$$

Slightly simplified equations for presentation:

$$
\begin{array}{lcl}
n'_1 & = & \emptyset = [1,1] \\
n_2 & = & \emptyset = [1,1] \\
n_3 & = & \emptyset = [1,1] \\
n'_3 & = & \emptyset \\
n_4 & = & \emptyset
\end{array}
\qquad
\begin{array}{cl}
= & [1,1] \\
= & n'_1 \cup n'_3 \\
= & n_2 \cap [-\infty, 999] \\
= & n_3 + 1 \\
= & n'_3 \cap [1000, +\infty]
\end{array}
$$

Slow Convergence

Because the previous example had no loops, all equations converged after one step. Compare to this example:

$$
\begin{array}{l}
[n := 1]^1 \\
\textbf{while } [n < 1000]^2 \textbf{ do} \\
\quad [n := n + 1]^3 \\
[\textbf{skip}]^4
\end{array}
$$

Slightly simplified equations for presentation:

$$
\begin{array}{lcllcl}
n'_1 & = & \emptyset = [1, 1] & & = & [1, 1] \\
n_2 & = & \emptyset = [1, 1] & & = & n'_1 \cup n'_3 \\
n_3 & = & \emptyset = [1, 1] & & = & n_2 \cap [-\infty, 999] \\
n'_3 & = & \emptyset = [2, 2] & & = & n_3 + 1 \\
n_4 & = & \emptyset & & = & n'_3 \cap [1000, +\infty]
\end{array}
$$

Slow Convergence

Because the previous example had no loops, all equations converged after one step. Compare to this example:

$$
\begin{array}{l}
[n := 1]^1 \\
\textbf{while } [n < 1000]^2 \textbf{ do} \\
\quad [n := n + 1]^3 \\
[\textbf{skip}]^4
\end{array}
$$

Slightly simplified equations for presentation:

$$
\begin{array}{lcl@{\qquad\qquad}cl}
n_1' & = & \emptyset = [1,1] & = & [1,1] \\
n_2 & = & \emptyset = [1,1] = [1,2] & = & n_1' \cup n_3' \\
n_3 & = & \emptyset = [1,1] & = & n_2 \cap [-\infty, 999] \\
n_3' & = & \emptyset = [2,2] & = & n_3 + 1 \\
n_4 & = & \emptyset & = & n_3' \cap [1000, +\infty]
\end{array}
$$

Slow Convergence

Because the previous example had no loops, all equations converged after one step. Compare to this example:

$$
\begin{array}{l}
[n := 1]^1 \\
\textbf{while } [n < 1000]^2 \textbf{ do} \\
\quad [n := n + 1]^3 \\
[\textbf{skip}]^4
\end{array}
$$

Slightly simplified equations for presentation:

$$
\begin{array}{lcllcl}
n'_1 & = & \emptyset = [1,1] & & = & [1,1] \\
n_2 & = & \emptyset = [1,1] = [1,2] & & = & n'_1 \cup n'_3 \\
n_3 & = & \emptyset = [1,1] = [1,2] & & = & n_2 \cap [-\infty, 999] \\
n'_3 & = & \emptyset = [2,2] & & = & n_3 + 1 \\
n_4 & = & \emptyset & & = & n'_3 \cap [1000, +\infty]
\end{array}
$$

Slow Convergence

Because the previous example had no loops, all equations converged after one step. Compare to this example:

$$
\begin{array}{l}
[n := 1]^1 \\
\textbf{while } [n < 1000]^2 \textbf{ do} \\
\quad [n := n + 1]^3 \\
[\textbf{skip}]^4
\end{array}
$$

Slightly simplified equations for presentation:

$$
\begin{array}{lclcl}
n'_1 & = & \emptyset = [1,1] & = & [1,1] \\
n_2 & = & \emptyset = [1,1] = [1,2] & = & n'_1 \cup n'_3 \\
n_3 & = & \emptyset = [1,1] = [1,2] & = & n_2 \cap [-\infty, 999] \\
n'_3 & = & \emptyset = [2,2] = [2,3] & = & n_3 + 1 \\
n_4 & = & \emptyset & = & n'_3 \cap [1000, +\infty]
\end{array}
$$

Slow Convergence

Because the previous example had no loops, all equations converged after one step. Compare to this example:

$$
\begin{array}{l}
[n := 1]^1 \\
\textbf{while } [n < 1000]^2 \textbf{ do} \\
\quad [n := n + 1]^3 \\
[\textbf{skip}]^4
\end{array}
$$

Slightly simplified equations for presentation:

$$
\begin{array}{lcllcl}
n'_1 & = & \emptyset = [1,1] & & = & [1,1] \\
n_2 & = & \emptyset = [1,1] = [1,2] = [1,3] & & = & n'_1 \cup n'_3 \\
n_3 & = & \emptyset = [1,1] = [1,2] & & = & n_2 \cap [-\infty, 999] \\
n'_3 & = & \emptyset = [2,2] = [2,3] & & = & n_3 + 1 \\
n_4 & = & \emptyset & & = & n'_3 \cap [1000, +\infty]
\end{array}
$$

Slow Convergence

Because the previous example had no loops, all equations converged after one step. Compare to this example:

$$
\begin{array}{l}
[n := 1]^1 \\
\textbf{while } [n < 1000]^2 \textbf{ do} \\
\quad [n := n + 1]^3 \\
[\textbf{skip}]^4
\end{array}
$$

Slightly simplified equations for presentation:

$$
\begin{array}{lcllcl}
n'_1 & = & \emptyset = [1,1] & & = & [1,1] \\
n_2 & = & \emptyset = [1,1] = [1,2] = [1,3] & & = & n'_1 \cup n'_3 \\
n_3 & = & \emptyset = [1,1] = [1,2] = [1,3] & & = & n_2 \cap [-\infty, 999] \\
n'_3 & = & \emptyset = [2,2] = [2,3] & & = & n_3 + 1 \\
n_4 & = & \emptyset & & = & n'_3 \cap [1000, +\infty]
\end{array}
$$

Slow Convergence

Because the previous example had no loops, all equations converged after one step. Compare to this example:

$$
\begin{array}{l}
[n := 1]^1 \\
\textbf{while } [n < 1000]^2 \textbf{ do} \\
\quad [n := n + 1]^3 \\
[\textbf{skip}]^4
\end{array}
$$

Slightly simplified equations for presentation:

$$
\begin{array}{lclcl}
n_1' & = & \emptyset = [1,1] & = & [1,1] \\
n_2 & = & \emptyset = [1,1] = [1,2] = [1,3] = \cdots & = & n_1' \cup n_3' \\
n_3 & = & \emptyset = [1,1] = [1,2] = [1,3] = \cdots & = & n_2 \cap [-\infty, 999] \\
n_3' & = & \emptyset = [2,2] = [2,3] = [2,4] = \cdots & = & n_3 + 1 \\
n_4 & = & \emptyset & = & n_3' \cap [1000, +\infty]
\end{array}
$$

This is going to take a long time to converge! (1000 steps)

Widening

Our interval abstraction is too detailed, making our loop iterations take ages.

Widening

Our interval abstraction is too detailed, making our loop iterations take ages.

Solution

Let n be the value we are updating and m be the result of the next iteration. Then, we update with $n \triangledown m$ instead of m:

$$\begin{array}{lcl}
\emptyset \triangledown m & = & m \\
n \triangledown \emptyset & = & n \\
[\ell_0, u_0] \triangledown [\ell_1, u_1] & = & [\textbf{if } \ell_1 < \ell_0 \textbf{ then } -\infty \textbf{ else } \ell_1 \\
 & & , \textbf{if } u_1 > u_0 \textbf{ then } +\infty \textbf{ else } u_1]
\end{array}$$

In other words, if we ever try to loosen a bound, we just extrapolate all the way to infinity.

Widening

Our interval abstraction is too detailed, making our loop iterations take ages.

Solution

Let n be the value we are updating and m be the result of the next iteration. Then, we update with $n \triangledown m$ instead of m:

$$
\begin{aligned}
\emptyset \triangledown m &= m \\
n \triangledown \emptyset &= n \\
[\ell_0, u_0] \triangledown [\ell_1, u_1] &= [\textbf{if } \ell_1 < \ell_0 \textbf{ then } -\infty \textbf{ else } \ell_1 \\
&\quad , \textbf{if } u_1 > u_0 \textbf{ then } +\infty \textbf{ else } u_1]
\end{aligned}
$$

In other words, if we ever try to loosen a bound, we just extrapolate all the way to infinity.

This is an overapproximation, but it converges much faster than the normal iterative sequence does.

Loops with Widening

$$
\begin{array}{lclcl}
n_1' & = & \emptyset & = & [1,1] \\
n_2 & = & \emptyset & = & n_1' \cup n_3' \\
n_3 & = & \emptyset & = & n_2 \cap [-\infty, 999] \\
n_3' & = & \emptyset & = & n_3 + 1 \\
n_4 & = & \emptyset & = & n_3' \cap [1000, +\infty]
\end{array}
$$

Loops with Widening

$$
\begin{aligned}
n_1' &= \emptyset = [1,1] & &= [1,1] \\
n_2 &= \emptyset & &= n_1' \cup n_3' \\
n_3 &= \emptyset & &= n_2 \cap [-\infty, 999] \\
n_3' &= \emptyset & &= n_3 + 1 \\
n_4 &= \emptyset & &= n_3' \cap [1000, +\infty]
\end{aligned}
$$

Loops with Widening

$$
\begin{array}{lclcl}
n_1' & = & \emptyset = [1,1] & = & [1,1] \\
n_2 & = & \emptyset = [1,1] & = & n_1' \cup n_3' \\
n_3 & = & \emptyset & = & n_2 \cap [-\infty, 999] \\
n_3' & = & \emptyset & = & n_3 + 1 \\
n_4 & = & \emptyset & = & n_3' \cap [1000, +\infty]
\end{array}
$$

Loops with Widening

$$
\begin{aligned}
n_1' &= \emptyset = [1,1] & &= [1,1]\\
n_2 &= \emptyset = [1,1] & &= n_1' \cup n_3'\\
n_3 &= \emptyset = [1,1] & &= n_2 \cap [-\infty, 999]\\
n_3' &= \emptyset & &= n_3 + 1\\
n_4 &= \emptyset & &= n_3' \cap [1000, +\infty]
\end{aligned}
$$

Loops with Widening

$$
\begin{aligned}
n'_1 &= \emptyset = [1,1] && = [1,1] \\
n_2 &= \emptyset = [1,1] && = n'_1 \cup n'_3 \\
n_3 &= \emptyset = [1,1] && = n_2 \cap [-\infty, 999] \\
n'_3 &= \emptyset = [2,2] && = n_3 + 1 \\
n_4 &= \emptyset && = n'_3 \cap [1000, +\infty]
\end{aligned}
$$

Loops with Widening

$$
\begin{aligned}
n'_1 &= \emptyset = [1,1] && = [1,1] \\
n_2 &= \emptyset = [1,1] = [1,+\infty] && = n'_1 \cup n'_3 \\
n_3 &= \emptyset = [1,1] && = n_2 \cap [-\infty, 999] \\
n'_3 &= \emptyset = [2,2] && = n_3 + 1 \\
n_4 &= \emptyset && = n'_3 \cap [1000, +\infty]
\end{aligned}
$$

Loops with Widening

$$\begin{array}{lcllcl}
n_1' & = & \emptyset = [1,1] & & = & [1,1] \\
n_2 & = & \emptyset = [1,1] = [1,+\infty] & & = & n_1' \cup n_3' \\
n_3 & = & \emptyset = [1,1] = [1,999] & & = & n_2 \cap [-\infty, 999] \\
n_3' & = & \emptyset = [2,2] & & = & n_3 + 1 \\
n_4 & = & \emptyset & & = & n_3' \cap [1000, +\infty]
\end{array}$$

Loops with Widening

$$
\begin{array}{lcll}
n'_1 & = & \emptyset = [1,1] & = [1,1] \\
n_2 & = & \emptyset = [1,1] = [1,+\infty] & = n'_1 \cup n'_3 \\
n_3 & = & \emptyset = [1,1] = [1,999] & = n_2 \cap [-\infty,999] \\
n'_3 & = & \emptyset = [2,2] = [2,1000] & = n_3 + 1 \\
n_4 & = & \emptyset & = n'_3 \cap [1000,+\infty]
\end{array}
$$

Loops with Widening

$$
\begin{array}{lclcl}
n_1' & = & \emptyset = [1,1] & = & [1,1] \\
n_2 & = & \emptyset = [1,1] = [1,+\infty] = [1,1000] & = & n_1' \cup n_3' \\
n_3 & = & \emptyset = [1,1] = [1,999] & = & n_2 \cap [-\infty,999] \\
n_3' & = & \emptyset = [2,2] = [2,1000] & = & n_3 + 1 \\
n_4 & = & \emptyset & = & n_3' \cap [1000,+\infty]
\end{array}
$$

Loops with Widening

$$
\begin{array}{lclcl}
n'_1 & = & \emptyset = [1,1] & = & [1,1] \\
n_2 & = & \emptyset = [1,1] = [1,+\infty] = [1,1000] & = & n'_1 \cup n'_3 \\
n_3 & = & \emptyset = [1,1] = [1,999] = [1,999] & = & n_2 \cap [-\infty, 999] \\
n'_3 & = & \emptyset = [2,2] = [2,1000] & = & n_3 + 1 \\
n_4 & = & \emptyset & = & n'_3 \cap [1000, +\infty]
\end{array}
$$

Loops with Widening

$$
\begin{array}{lcll}
n_1' & = & \emptyset = [1,1] & = \quad [1,1] \\
n_2 & = & \emptyset = [1,1] = [1,+\infty] = [1,1000] & = \quad n_1' \cup n_3' \\
n_3 & = & \emptyset = [1,1] = [1,999] = [1,999] & = \quad n_2 \cap [-\infty, 999] \\
n_3' & = & \emptyset = [2,2] = [2,1000] = [2,1000] & = \quad n_3 + 1 \\
n_4 & = & \emptyset & = \quad n_3' \cap [1000, +\infty]
\end{array}
$$

Loops with Widening

$$
\begin{array}{lllll}
n_1' & = & \emptyset = [1,1] & = & [1,1] \\
n_2 & = & \emptyset = [1,1] = [1,+\infty] = [1,1000] & = & n_1' \cup n_3' \\
n_3 & = & \emptyset = [1,1] = [1,999] = [1,999] & = & n_2 \cap [-\infty,999] \\
n_3' & = & \emptyset = [2,2] = [2,1000] = [2,1000] & = & n_3 + 1 \\
n_4 & = & \emptyset = [1000,1000] & = & n_3' \cap [1000,+\infty]
\end{array}
$$

Beyond Interval Anaylsis

Interval analysis is very effective but not very accurate, because it doesn't express the relationships between variables.

Beyond Interval Anaylsis

Interval analysis is very effective but not very accurate, because it doesn't express the relationships between variables.

Predicate abstraction and *polyhedral models* do better in many cases, but are more complicated.

Beyond Interval Anaylsis

Interval analysis is very effective but not very accurate, because it doesn't express the relationships between variables.
Predicate abstraction and *polyhedral models* do better in many cases, but are more complicated.
All are based on the same principle of least-fixed point of a system of equations.

Data-flow Analysis

Data-flow analysis is a type of static analysis used extensively in compilers.

Data-flow Analysis

Data-flow analysis is a type of static analysis used extensively in compilers.

Example

- *Available Expressions Analysis* – Compute what expressions **must** have already been computed (and don't need to be recomputed).

Data-flow Analysis

Data-flow analysis is a type of static analysis used extensively in compilers.

Example

- *Available Expressions Analysis* – Compute what expressions **must** have already been computed (and don't need to be recomputed).
- *Live Variables Analysis* – Compute which variables **may** be read before next being written to (and thus hold important values).

Data-flow Analysis

Data-flow analysis is a type of static analysis used extensively in compilers.

Example

- *Available Expressions Analysis* – Compute what expressions **must** have already been computed (and don't need to be recomputed).
- *Live Variables Analysis* – Compute which variables **may** be read before next being written to (and thus hold important values).

Data-flow analyses may be *forwards* or *backwards*, and *may* or *must*.

Data-flow Analysis

Data-flow analysis is a type of static analysis used extensively in compilers.

Example

- *Available Expressions Analysis* – Compute what expressions **must** have already been computed (and don't need to be recomputed).
- *Live Variables Analysis* – Compute which variables **may** be read before next being written to (and thus hold important values).

Data-flow analyses may be *forwards* or *backwards*, and *may* or *must*.
AEA is a forwards must analysis.

Data-flow Analysis

Data-flow analysis is a type of static analysis used extensively in compilers.

Example

- *Available Expressions Analysis* – Compute what expressions **must** have already been computed (and don't need to be recomputed).
- *Live Variables Analysis* – Compute which variables **may** be read before next being written to (and thus hold important values).

Data-flow analyses may be *forwards* or *backwards*, and *may* or *must*.
AEA is a forwards must analysis. LVA is a backwards may analysis.

Step 1: Gen and Kill

Each location in the CFG has an associated gen set, of generated information, and kill set, of information that is no longer accurate.

Step 1: Gen and Kill

Each location in the CFG has an associated gen set, of generated information, and kill set, of information that is no longer accurate.

Example (AEA)

In AEA, $\text{gen}_{AE}(\ell)$ is the expressions evaluated (and not updated) in ℓ and $\text{kill}_{AE}(\ell)$ is those expressions updated by ℓ.

Step 1: Gen and Kill

Each location in the CFG has an associated gen set, of generated information, and kill set, of information that is no longer accurate.

Example (AEA)

In AEA, $\text{gen}_{AE}(\ell)$ is the expressions evaluated (and not updated) in ℓ and $\text{kill}_{AE}(\ell)$ is those expressions updated by ℓ.
For example, $x := a + b$ would generate $\{a + b\}$, but kill any expression involving x.

Step 1: Gen and Kill

Each location in the CFG has an associated gen set, of generated information, and kill set, of information that is no longer accurate.

Example (AEA)

In AEA, $\text{gen}_{AE}(\ell)$ is the expressions evaluated (and not updated) in ℓ and $\text{kill}_{AE}(\ell)$ is those expressions updated by ℓ.
For example, $x := a + b$ would generate $\{a + b\}$, but kill any expression involving x.
Note: $a := a + 1$ would kill $a + 1$, not generate it. Why?

Example (LVA)

In LVA

Step 1: Gen and Kill

Each location in the CFG has an associated gen set, of generated information, and kill set, of information that is no longer accurate.

Example (AEA)

In AEA, $\text{gen}_{AE}(\ell)$ is the expressions evaluated (and not updated) in ℓ and $\text{kill}_{AE}(\ell)$ is those expressions updated by ℓ.
For example, $x := a + b$ would generate $\{a + b\}$, but kill any expression involving x.
Note: $a := a + 1$ would kill $a + 1$, not generate it. Why?

Example (LVA)

In LVA, $\text{gen}_{LV}(\ell)$ is the variables read (and not written to) in ℓ and $\text{kill}_{LV}(\ell)$ would be the variables written to in ℓ.
For example, $x := a + b$ would generate $\{a, b\}$ and kill $\{x\}$.

Available Expressions Example

$$
\begin{tikzcd}
{[x := a + b]^1} \arrow[d] \\
{[y := a * b]^2} \arrow[d] \\
{[y > a + b]^3} \arrow[d] \arrow[r] & {} \\
{[a := a + 1]^4} \arrow[d] \\
{[x := a + b]^5} \arrow[uu, bend left]
\end{tikzcd}
$$

ℓ	gen_{AE}	kill_{AE}
1		

Available Expressions Example

$$
\begin{tikzcd}
{[x := a + b]^1} \arrow[d] \\
{[y := a * b]^2} \arrow[d] \\
{[y > a + b]^3} \arrow[d] \arrow[r] & {} \\
{[a := a + 1]^4} \arrow[d] \\
{[x := a + b]^5} \arrow[uu, bend left=60]
\end{tikzcd}
$$

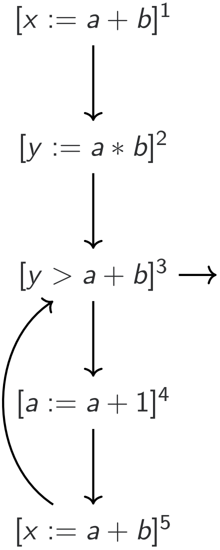
ℓ	gen_{AE}	kill_{AE}
1	$\{a + b\}$	

Available Expressions Example

$$
\begin{tikzcd}
{[x := a + b]^1} \arrow[d] & \\
{[y := a * b]^2} \arrow[d] & \\
{[y > a + b]^3} \arrow[d] \arrow[r] & {} \\
{[a := a + 1]^4} \arrow[d] & \\
{[x := a + b]^5} \arrow[uu, bend left] &
\end{tikzcd}
$$

ℓ	gen_{AE}	kill_{AE}
1	$\{a + b\}$	$\emptyset$
2		

Available Expressions Example



ℓ	gen_{AE}	kill_{AE}
1	$\{a + b\}$	$\emptyset$
2	$\{a * b\}$	$\emptyset$
3		

Available Expressions Example

$$
\begin{tikzcd}
{[x := a + b]^1} \arrow[d] \\
{[y := a * b]^2} \arrow[d] \\
{[y > a + b]^3} \arrow[d] \arrow[r] & {} \\
{[a := a + 1]^4} \arrow[d] \\
{[x := a + b]^5} \arrow[uu, bend left]
\end{tikzcd}
$$

ℓ	gen_{AE}	kill_{AE}
1	$\{a + b\}$	$\emptyset$
2	$\{a * b\}$	$\emptyset$
3	$\{a + b\}$	$\emptyset$
4		

Available Expressions Example

$$
\begin{tikzcd}
{[x := a + b]^1} \arrow[d] \\
{[y := a * b]^2} \arrow[d] \\
{[y > a + b]^3} \arrow[d] \arrow[r] & {} \\
{[a := a + 1]^4} \arrow[d] \\
{[x := a + b]^5} \arrow[uu, bend left]
\end{tikzcd}
$$

ℓ	gen_{AE}	kill_{AE}
1	$\{a + b\}$	$\emptyset$
2	$\{a * b\}$	$\emptyset$
3	$\{a + b\}$	$\emptyset$
4	$\emptyset$	

Available Expressions Example

$$
\begin{tikzcd}
{[x := a + b]^1} \arrow[d] \\
{[y := a * b]^2} \arrow[d] \\
{[y > a + b]^3} \arrow[d] \arrow[r] & {} \\
{[a := a + 1]^4} \arrow[d] \\
{[x := a + b]^5} \arrow[uu, bend left]
\end{tikzcd}
$$

ℓ	gen_{AE}	kill_{AE}
1	$\{a + b\}$	$\emptyset$
2	$\{a * b\}$	$\emptyset$
3	$\{a + b\}$	$\emptyset$
4	$\emptyset$	$\{a + b, a * b, a + 1\}$
5		

Available Expressions Example

$$
\begin{tikzcd}
{[x := a + b]^1} \arrow[d] \\
{[y := a * b]^2} \arrow[d] \\
{[y > a + b]^3} \arrow[d] \arrow[r] & {} \\
{[a := a + 1]^4} \arrow[d] \\
{[x := a + b]^5} \arrow[uu, bend left]
\end{tikzcd}
$$

ℓ	gen_{AE}	kill_{AE}
1	$\{a + b\}$	$\emptyset$
2	$\{a * b\}$	$\emptyset$
3	$\{a + b\}$	$\emptyset$
4	$\emptyset$	$\{a + b, a * b, a + 1\}$
5	$\{a + b\}$	$\emptyset$

What to do now?

Specify an equation system that relates these, then find the fixed point!

Forwards Must Analysis

The set of available expressions as we enter a location ℓ is the intersection of all available expressions from the predecessors to ℓ:

$$AE_{\text{entry}}(\ell) = \begin{cases} \emptyset & \text{if } \ell \in I_{\text{CFG}} \\ \bigcap\{AE_{\text{exit}}(\ell') \mid (\ell', \ell) \in \delta_{\text{CFG}}\} & \text{otherwise} \end{cases}$$

Forwards Must Analysis

The set of available expressions as we enter a location ℓ is the intersection of all available expressions from the predecessors to ℓ:

$$AE_{\text{entry}}(\ell) = \begin{cases} \emptyset & \text{if } \ell \in I_{\text{CFG}} \\ \bigcap\{AE_{\text{exit}}(\ell') \mid (\ell', \ell) \in \delta_{\text{CFG}}\} & \text{otherwise} \end{cases}$$

We choose intersection because we want expressions that are **definitely** available no matter what route we took to get to ℓ (a **must** analysis).

Forwards Must Analysis

The set of available expressions as we enter a location ℓ is the intersection of all available expressions from the predecessors to ℓ:

$$AE_{\text{entry}}(\ell) = \begin{cases} \emptyset & \text{if } \ell \in I_{\text{CFG}} \\ \bigcap\{AE_{\text{exit}}(\ell') \mid (\ell', \ell) \in \delta_{\text{CFG}}\} & \text{otherwise} \end{cases}$$

We choose intersection because we want expressions that are **definitely** available no matter what route we took to get to ℓ (a **must** analysis).

The set of available expressions as we exit a location ℓ is the set that we entered with, minus anything we kill, plus anything we generate:

$$AE_{\text{exit}}(\ell) = (AE_{\text{entry}}(\ell) \setminus \text{kill}_{\text{AE}}(\ell)) \cup \text{gen}_{\text{AE}}(\ell)$$

Example for AEA

$$
\begin{tikzcd}
{[x := a + b]^1} \arrow[d] \\
{[y := a * b]^2} \arrow[d] \\
{[y > a + b]^3} \arrow[d] \\
{[a := a + 1]^4} \arrow[d] \\
{[x := a + b]^5} \arrow[uu, bend left]
\end{tikzcd}
$$

ℓ	gen_{AE}	kill_{AE}
1	$\{a + b\}$	$\emptyset$
2	$\{a * b\}$	$\emptyset$
3	$\{a + b\}$	$\emptyset$
4	$\emptyset$	$\{a + b, a * b, a + 1\}$
5	$\{a + b\}$	

ℓ	$AE_{\text{entry}}(\ell)$	$AE_{\text{exit}}(\ell)$
1	$\emptyset$	

Example for AEA

$$
\begin{tikzcd}
{[x := a+b]^1} \arrow[d] \\
{[y := a*b]^2} \arrow[d] \\
{[y > a+b]^3} \arrow[d] \arrow[r] & {} \\
{[a := a+1]^4} \arrow[d] \\
{[x := a+b]^5} \arrow[uu, bend left=60]
\end{tikzcd}
$$

ℓ	gen_{AE}	$kill_{AE}$
1	$\{a+b\}$	$\emptyset$
2	$\{a*b\}$	$\emptyset$
3	$\{a+b\}$	$\emptyset$
4	$\emptyset$	$\{a+b, a*b, a+1\}$
5	$\{a+b\}$	

ℓ	$AE_{entry}(\ell)$	$AE_{exit}(\ell)$
1	$\emptyset$	$AE_{entry}(1) \cup \{a+b\}$

Example for AEA

$$
\begin{tikzcd}
{[x := a+b]^1} \arrow[d] \\
{[y := a*b]^2} \arrow[d] \\
{[y > a+b]^3} \arrow[d] \arrow[r] & {} \\
{[a := a+1]^4} \arrow[d] \\
{[x := a+b]^5} \arrow[uu, bend left]
\end{tikzcd}
$$

ℓ	gen_{AE}	$kill_{AE}$
1	$\{a+b\}$	$\emptyset$
2	$\{a*b\}$	$\emptyset$
3	$\{a+b\}$	$\emptyset$
4	$\emptyset$	$\{a+b, a*b, a+1\}$
5	$\{a+b\}$	

ℓ	$AE_{entry}(\ell)$	$AE_{exit}(\ell)$
1	$\emptyset$	$AE_{entry}(1) \cup \{a+b\}$
2	$AE_{exit}(1)$	

Example for AEA

$$
\begin{tikzcd}
{[x := a + b]^1} \arrow[d] \\
{[y := a * b]^2} \arrow[d] \\
{[y > a + b]^3} \arrow[d] \arrow[r] & {} \\
{[a := a + 1]^4} \arrow[d] \\
{[x := a + b]^5} \arrow[uu, bend left]
\end{tikzcd}
$$

ℓ	gen_{AE}	kill_{AE}
1	$\{a + b\}$	$\emptyset$
2	$\{a * b\}$	$\emptyset$
3	$\{a + b\}$	$\emptyset$
4	$\emptyset$	$\{a + b, a * b, a + 1\}$
5	$\{a + b\}$	

ℓ	$AE_{\text{entry}}(\ell)$	$AE_{\text{exit}}(\ell)$
1	$\emptyset$	$AE_{\text{entry}}(1) \cup \{a + b\}$
2	$AE_{\text{exit}}(1)$	$AE_{\text{entry}}(2) \cup \{a * b\}$

Example for AEA

$$
\begin{tikzcd}
{[x := a + b]^1} \arrow[d] \\
{[y := a * b]^2} \arrow[d] \\
{[y > a + b]^3} \arrow[d] \arrow[r] & {} \\
{[a := a + 1]^4} \arrow[d] \\
{[x := a + b]^5} \arrow[uu, bend left]
\end{tikzcd}
$$

ℓ	gen_{AE}	kill_{AE}
1	$\{a + b\}$	$\emptyset$
2	$\{a * b\}$	$\emptyset$
3	$\{a + b\}$	$\emptyset$
4	$\emptyset$	$\{a + b, a * b, a + 1\}$
5	$\{a + b\}$	

ℓ	$AE_{\text{entry}}(\ell)$	$AE_{\text{exit}}(\ell)$
1	$\emptyset$	$AE_{\text{entry}}(1) \cup \{a + b\}$
2	$AE_{\text{exit}}(1)$	$AE_{\text{entry}}(2) \cup \{a * b\}$
3	$AE_{\text{exit}}(2) \cap AE_{\text{exit}}(5)$	

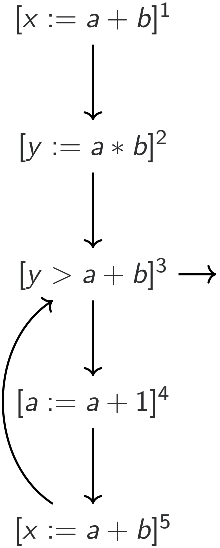
Example for AEA

$$
\begin{tikzcd}
{[x := a + b]^1} \arrow[d] \\
{[y := a * b]^2} \arrow[d] \\
{[y > a + b]^3} \arrow[d] \arrow[r] & {} \\
{[a := a + 1]^4} \arrow[d] \\
{[x := a + b]^5} \arrow[uu, bend left]
\end{tikzcd}
$$

ℓ	gen_{AE}	$kill_{AE}$
1	$\{a+b\}$	$\emptyset$
2	$\{a*b\}$	$\emptyset$
3	$\{a+b\}$	$\emptyset$
4	$\emptyset$	$\{a+b, a*b, a+1\}$
5	$\{a+b\}$	

ℓ	$AE_{entry}(\ell)$	$AE_{exit}(\ell)$
1	$\emptyset$	$AE_{entry}(1) \cup \{a+b\}$
2	$AE_{exit}(1)$	$AE_{entry}(2) \cup \{a*b\}$
3	$AE_{exit}(2) \cap AE_{exit}(5)$	$AE_{entry}(3) \cup \{a+b\}$

Example for AEA



ℓ	gen_{AE}	kill_{AE}
1	$\{a + b\}$	$\emptyset$
2	$\{a * b\}$	$\emptyset$
3	$\{a + b\}$	$\emptyset$
4	$\emptyset$	$\{a + b, a * b, a + 1\}$
5	$\{a + b\}$	

ℓ	$AE_{\mathrm{entry}}(\ell)$	$AE_{\mathrm{exit}}(\ell)$
1	$\emptyset$	$AE_{\mathrm{entry}}(1) \cup \{a + b\}$
2	$AE_{\mathrm{exit}}(1)$	$AE_{\mathrm{entry}}(2) \cup \{a * b\}$
3	$AE_{\mathrm{exit}}(2) \cap AE_{\mathrm{exit}}(5)$	$AE_{\mathrm{entry}}(3) \cup \{a + b\}$
4	$AE_{\mathrm{exit}}(3)$	

Example for AEA

$$
\begin{tikzcd}
{[x := a + b]^1} \arrow[d] \\
{[y := a * b]^2} \arrow[d] \\
{[y > a + b]^3} \arrow[d] \arrow[r] & {} \\
{[a := a + 1]^4} \arrow[d] \\
{[x := a + b]^5} \arrow[uu, bend left]
\end{tikzcd}
$$

ℓ	gen_{AE}	$kill_{AE}$
1	$\{a+b\}$	$\emptyset$
2	$\{a*b\}$	$\emptyset$
3	$\{a+b\}$	$\emptyset$
4	$\emptyset$	$\{a+b, a*b, a+1\}$
5	$\{a+b\}$	

ℓ	$AE_{entry}(\ell)$	$AE_{exit}(\ell)$
1	$\emptyset$	$AE_{entry}(1) \cup \{a+b\}$
2	$AE_{exit}(1)$	$AE_{entry}(2) \cup \{a*b\}$
3	$AE_{exit}(2) \cap AE_{exit}(5)$	$AE_{entry}(3) \cup \{a+b\}$
4	$AE_{exit}(3)$	$AE_{entry}(4) \setminus kill_{AE}(4)$

Example for AEA

$$
\begin{tikzcd}
{[x := a + b]^1} \arrow[d] \\
{[y := a * b]^2} \arrow[d] \\
{[y > a + b]^3} \arrow[d] \\
{[a := a + 1]^4} \arrow[d] \\
{[x := a + b]^5} \arrow[uu, bend left]
\end{tikzcd}
\longrightarrow
$$

ℓ	gen_{AE}	kill_{AE}
1	$\{a+b\}$	$\emptyset$
2	$\{a*b\}$	$\emptyset$
3	$\{a+b\}$	$\emptyset$
4	$\emptyset$	$\{a+b, a*b, a+1\}$
5	$\{a+b\}$	

ℓ	$AE_{\text{entry}}(\ell)$	$AE_{\text{exit}}(\ell)$
1	$\emptyset$	$AE_{\text{entry}}(1) \cup \{a+b\}$
2	$AE_{\text{exit}}(1)$	$AE_{\text{entry}}(2) \cup \{a*b\}$
3	$AE_{\text{exit}}(2) \cap AE_{\text{exit}}(5)$	$AE_{\text{entry}}(3) \cup \{a+b\}$
4	$AE_{\text{exit}}(3)$	$AE_{\text{entry}}(4) \setminus \text{kill}_{AE}(4)$
5	$AE_{\text{exit}}(4)$	

Example for AEA

$$
\begin{tikzcd}
{[x := a + b]^1} \arrow[d] \\
{[y := a * b]^2} \arrow[d] \\
{[y > a + b]^3} \arrow[d] \arrow[r] & {} \\
{[a := a + 1]^4} \arrow[d] \\
{[x := a + b]^5} \arrow[uu, bend left]
\end{tikzcd}
$$

ℓ	gen_{AE}	$kill_{AE}$
1	$\{a + b\}$	$\emptyset$
2	$\{a * b\}$	$\emptyset$
3	$\{a + b\}$	$\emptyset$
4	$\emptyset$	$\{a + b, a * b, a + 1\}$
5	$\{a + b\}$	

ℓ	$AE_{\text{entry}}(\ell)$	$AE_{\text{exit}}(\ell)$
1	$\emptyset$	$AE_{\text{entry}}(1) \cup \{a + b\}$
2	$AE_{\text{exit}}(1)$	$AE_{\text{entry}}(2) \cup \{a * b\}$
3	$AE_{\text{exit}}(2) \cap AE_{\text{exit}}(5)$	$AE_{\text{entry}}(3) \cup \{a + b\}$
4	$AE_{\text{exit}}(3)$	$AE_{\text{entry}}(4) \setminus kill_{AE}(4)$
5	$AE_{\text{exit}}(4)$	$AE_{\text{entry}}(5) \cup \{a + b\}$

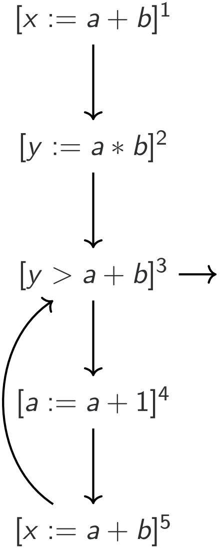
Results

$$
\begin{tikzcd}
{[x := a + b]^1} \arrow[d] \\
{[y := a * b]^2} \arrow[d] \\
{[y > a + b]^3} \arrow[d] \arrow[r] & {} \\
{[a := a + 1]^4} \arrow[d] \\
{[x := a + b]^5} \arrow[uu, bend left]
\end{tikzcd}
$$

ℓ	gen_{AE}	kill_{AE}
1	$\{a + b\}$	$\emptyset$
2	$\{a * b\}$	$\emptyset$
3	$\{a + b\}$	$\emptyset$
4	$\emptyset$	$\{a + b, a * b, a + 1\}$
5	$\{a + b\}$	

ℓ	$AE_{\text{entry}}(\ell)$	$AE_{\text{exit}}(\ell)$
1	$\emptyset$	$\{a + b\}$
2	$\{a + b\}$	$\{a + b, a * b\}$
3	$\{a + b\}$	$\{a + b\}$
4	$\{a + b\}$	$\emptyset$
5	$\emptyset$	$\{a + b\}$

Results



ℓ	gen_{AE}	$kill_{AE}$
1	$\{a + b\}$	$\emptyset$
2	$\{a * b\}$	$\emptyset$
3	$\{a + b\}$	$\emptyset$
4	$\emptyset$	$\{a + b, a * b, a + 1\}$
5	$\{a + b\}$	

ℓ	$AE_{entry}(\ell)$	$AE_{exit}(\ell)$
1	$\emptyset$	$\{a + b\}$
2	$\{a + b\}$	$\{a + b, a * b\}$
3	$\{a + b\}$	$\{a + b\}$
4	$\{a + b\}$	$\emptyset$
5	$\emptyset$	$\{a + b\}$

Note $\ell = 3$ can be optimised, as $a + b$ is already computed!

Backwards May Analysis

The set of live variables as we exit a location ℓ is the union of live variables from the successors to ℓ:

$$LV_{\text{exit}}(\ell) = \begin{cases} \emptyset & \text{if } \ell \in F_{\text{CFG}} \\ \bigcup\{LV_{\text{entry}}(\ell') \mid (\ell, \ell') \in \delta_{\text{CFG}}\} & \text{otherwise} \end{cases}$$

Backwards May Analysis

The set of live variables as we exit a location ℓ is the union of live variables from the successors to ℓ:

$$LV_{\text{exit}}(\ell) = \begin{cases} \emptyset & \text{if } \ell \in F_{\mathsf{CFG}} \\ \bigcup\{LV_{\text{entry}}(\ell') \mid (\ell, \ell') \in \delta_{\mathsf{CFG}}\} & \text{otherwise} \end{cases}$$

We choose union because we want any variables that **might be** used in a successor to be marked live in ℓ (a **may** analysis).

Backwards May Analysis

The set of live variables as we exit a location ℓ is the union of live variables from the successors to ℓ:

$$LV_{\text{exit}}(\ell) = \begin{cases} \emptyset & \text{if } \ell \in F_{\text{CFG}} \\ \bigcup\{LV_{\text{entry}}(\ell') \mid (\ell, \ell') \in \delta_{\text{CFG}}\} & \text{otherwise} \end{cases}$$

We choose union because we want any variables that **might be** used in a successor to be marked live in ℓ (a **may** analysis).

The set of live variables as we enter a location ℓ is the set of live variables at the exit, minus anything we kill (write to), plus anything we generate (read from):

$$LV_{\text{entry}}(\ell) = (LV_{\text{exit}}(\ell) \setminus \text{kill}_{\text{LV}}(\ell)) \cup \text{gen}_{\text{LV}}(\ell)$$

Backwards May Analysis

The set of live variables as we exit a location ℓ is the union of live variables from the successors to ℓ:

$$LV_{\text{exit}}(\ell) = \begin{cases} \emptyset & \text{if } \ell \in F_{\text{CFG}} \\ \bigcup\{LV_{\text{entry}}(\ell') \mid (\ell, \ell') \in \delta_{\text{CFG}}\} & \text{otherwise} \end{cases}$$

We choose union because we want any variables that **might be** used in a successor to be marked live in ℓ (a **may** analysis).

The set of live variables as we enter a location ℓ is the set of live variables at the exit, minus anything we kill (write to), plus anything we generate (read from):

$$LV_{\text{entry}}(\ell) = (LV_{\text{exit}}(\ell) \setminus \text{kill}_{\text{LV}}(\ell)) \cup \text{gen}_{\text{LV}}(\ell)$$

The entry and exit equations are flipped because this is a backwards analysis.

Example for LVA

$$
\begin{tikzcd}
{[a := 0]^1} \arrow[d] \\
{[b := a * 1]^2} \arrow[d] \\
{[c := c + b]^3} \arrow[d] \\
{[a := b * 2]^4} \arrow[d] \\
{[a < 9]^5} \arrow[r] \arrow[uuu, bend left] & {[\textbf{return } c]^6}
\end{tikzcd}
$$

ℓ	gen_{LV}	kill_{LV}
1		

Example for LVA

$$
\begin{tikzcd}
{[a := 0]^1} \arrow[d] \\
{[b := a * 1]^2} \arrow[d] \\
{[c := c + b]^3} \arrow[d] \\
{[a := b * 2]^4} \arrow[d] \\
{[a < 9]^5} \arrow[r] \arrow[uuu, bend left] & {[\textbf{return } c]^6}
\end{tikzcd}
$$

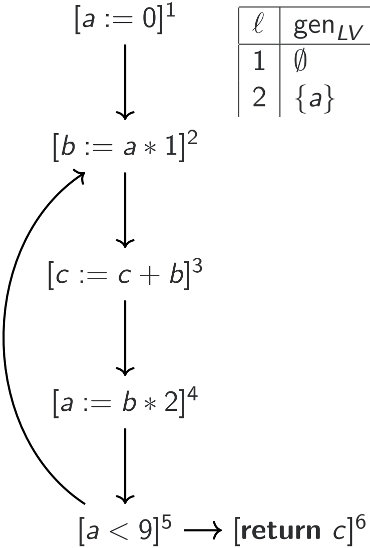
ℓ	gen_{LV}	kill_{LV}
1	$\emptyset$	

Example for LVA

$$
\begin{tikzcd}
{[a := 0]^1} \arrow[d] & \\
{[b := a * 1]^2} \arrow[d] & \\
{[c := c + b]^3} \arrow[d] & \\
{[a := b * 2]^4} \arrow[d] & \\
{[a < 9]^5} \arrow[r] \arrow[uuu, bend left] & {[\textbf{return } c]^6}
\end{tikzcd}
$$

ℓ	gen_{LV}	kill_{LV}
1	$\emptyset$	$\{a\}$
2		

Example for LVA



ℓ	gen_{LV}	kill_{LV}
1	$\emptyset$	$\{a\}$
2	$\{a\}$	

Example for LVA

$$
\begin{tikzcd}
{[a := 0]^1} \arrow[d] & \\
{[b := a * 1]^2} \arrow[d] & \\
{[c := c + b]^3} \arrow[d] & \\
{[a := b * 2]^4} \arrow[d] & \\
{[a < 9]^5} \arrow[uuu, bend left=60] \arrow[r] & {[\textbf{return } c]^6}
\end{tikzcd}
$$

ℓ	gen_{LV}	kill_{LV}
1	$\emptyset$	$\{a\}$
2	$\{a\}$	$\{b\}$
3		

Example for LVA

$$
\begin{tikzcd}
{[a := 0]^1} \arrow[d] & \\
{[b := a * 1]^2} \arrow[d] & \\
{[c := c + b]^3} \arrow[d] & \\
{[a := b * 2]^4} \arrow[d] & \\
{[a < 9]^5} \arrow[r] \arrow[uuu, bend left=60] & {[\textbf{return } c]^6}
\end{tikzcd}
$$

ℓ	gen_{LV}	kill_{LV}
1	$\emptyset$	$\{a\}$
2	$\{a\}$	$\{b\}$
3	$\{b\}$	$\{c\}$
4		

Example for LVA

$$
\begin{tikzcd}
{[a := 0]^1} \arrow[d] & \\
{[b := a * 1]^2} \arrow[d] & \\
{[c := c + b]^3} \arrow[d] & \\
{[a := b * 2]^4} \arrow[d] & \\
{[a < 9]^5} \arrow[r] \arrow[uuu, bend left=60] & {[\textbf{return } c]^6}
\end{tikzcd}
$$

ℓ	gen_{LV}	kill_{LV}
1	$\emptyset$	$\{a\}$
2	$\{a\}$	$\{b\}$
3	$\{b\}$	$\{c\}$
4	$\{b\}$	$\{a\}$
5		

Example for LVA

$$
\begin{tikzcd}
{[a := 0]^1} \arrow[d] & \\
{[b := a * 1]^2} \arrow[d] & \\
{[c := c + b]^3} \arrow[d] & \\
{[a := b * 2]^4} \arrow[d] & \\
{[a < 9]^5} \arrow[r] \arrow[uuu, bend left] & {[\textbf{return } c]^6}
\end{tikzcd}
$$

ℓ	gen_{LV}	kill_{LV}
1	$\emptyset$	$\{a\}$
2	$\{a\}$	$\{b\}$
3	$\{b\}$	$\{c\}$
4	$\{b\}$	$\{a\}$
5	$\{a\}$	$\emptyset$

ℓ	$LV_{\text{entry}}(\ell)$	$LV_{\text{exit}}(\ell)$
1		

Example for LVA

$$
\begin{tikzcd}
{[a := 0]^1} \arrow[d] & \\
{[b := a * 1]^2} \arrow[d] & \\
{[c := c + b]^3} \arrow[d] & \\
{[a := b * 2]^4} \arrow[d] & \\
{[a < 9]^5} \arrow[r] \arrow[uuu, bend left] & {[\textbf{return } c]^6}
\end{tikzcd}
$$

ℓ	gen_{LV}	kill_{LV}
1	$\emptyset$	$\{a\}$
2	$\{a\}$	$\{b\}$
3	$\{b\}$	$\{c\}$
4	$\{b\}$	$\{a\}$
5	$\{a\}$	$\emptyset$

ℓ	$LV_{\text{entry}}(\ell)$	$LV_{\text{exit}}(\ell)$
1	$LV_{\text{exit}}(1) \setminus \{a\}$	

Example for LVA

$$\begin{tikzcd}
{[a := 0]^1} \arrow[d] \\
{[b := a * 1]^2} \arrow[d] \\
{[c := c + b]^3} \arrow[d] \\
{[a := b * 2]^4} \arrow[d] \\
{[a < 9]^5} \arrow[r] \arrow[uuu, bend left] & {[\textbf{return } c]^6}
\end{tikzcd}$$

ℓ	gen_{LV}	kill_{LV}
1	$\emptyset$	$\{a\}$
2	$\{a\}$	$\{b\}$
3	$\{b\}$	$\{c\}$
4	$\{b\}$	$\{a\}$
5	$\{a\}$	$\emptyset$

ℓ	$LV_{\text{entry}}(\ell)$	$LV_{\text{exit}}(\ell)$
1	$LV_{\text{exit}}(1) \setminus \{a\}$	$LV_{\text{entry}}(2)$
2		

Example for LVA

$$
\begin{tikzcd}
{[a := 0]^1} \arrow[d] \\
{[b := a * 1]^2} \arrow[d] \\
{[c := c + b]^3} \arrow[d] \\
{[a := b * 2]^4} \arrow[d] \\
{[a < 9]^5} \arrow[uuu, bend left] \arrow[r] & {[\textbf{return } c]^6}
\end{tikzcd}
$$

ℓ	gen_{LV}	kill_{LV}
1	$\emptyset$	$\{a\}$
2	$\{a\}$	$\{b\}$
3	$\{b\}$	$\{c\}$
4	$\{b\}$	$\{a\}$
5	$\{a\}$	$\emptyset$

ℓ	$LV_{\text{entry}}(\ell)$	$LV_{\text{exit}}(\ell)$
1	$LV_{\text{exit}}(1) \setminus \{a\}$	$LV_{\text{entry}}(2)$
2	$LV_{\text{exit}}(2) \setminus \{b\} \cup \{a\}$	

Example for LVA

$$
\begin{tikzcd}
{[a := 0]^1} \arrow[d] \\
{[b := a * 1]^2} \arrow[d] \\
{[c := c + b]^3} \arrow[d] \\
{[a := b * 2]^4} \arrow[d] \\
{[a < 9]^5} \arrow[r] \arrow[uuu, bend left] & {[\textbf{return } c]^6}
\end{tikzcd}
$$

ℓ	gen_{LV}	kill_{LV}
1	$\emptyset$	$\{a\}$
2	$\{a\}$	$\{b\}$
3	$\{b\}$	$\{c\}$
4	$\{b\}$	$\{a\}$
5	$\{a\}$	$\emptyset$

ℓ	$LV_{\text{entry}}(\ell)$	$LV_{\text{exit}}(\ell)$
1	$LV_{\text{exit}}(1) \setminus \{a\}$	$LV_{\text{entry}}(2)$
2	$LV_{\text{exit}}(2) \setminus \{b\} \cup \{a\}$	$LV_{\text{entry}}(3)$
3		

Example for LVA

$$
\begin{tikzcd}
{[a := 0]^1} \arrow[d] \\
{[b := a * 1]^2} \arrow[d] \\
{[c := c + b]^3} \arrow[d] \\
{[a := b * 2]^4} \arrow[d] \\
{[a < 9]^5} \arrow[r] \arrow[uuu, bend left] & {[\textbf{return } c]^6}
\end{tikzcd}
$$

ℓ	gen_{LV}	kill_{LV}
1	$\emptyset$	$\{a\}$
2	$\{a\}$	$\{b\}$
3	$\{b\}$	$\{c\}$
4	$\{b\}$	$\{a\}$
5	$\{a\}$	$\emptyset$

ℓ	$LV_{\text{entry}}(\ell)$	$LV_{\text{exit}}(\ell)$
1	$LV_{\text{exit}}(1) \setminus \{a\}$	$LV_{\text{entry}}(2)$
2	$LV_{\text{exit}}(2) \setminus \{b\} \cup \{a\}$	$LV_{\text{entry}}(3)$
3	$LV_{\text{exit}}(3) \setminus \{c\} \cup \{b\}$	

Example for LVA

$$
\begin{tikzcd}
{[a := 0]^1} \arrow[d] & \\
{[b := a * 1]^2} \arrow[d] & \\
{[c := c + b]^3} \arrow[d] & \\
{[a := b * 2]^4} \arrow[d] & \\
{[a < 9]^5} \arrow[r] \arrow[uuu, bend left=60] & {[\textbf{return } c]^6}
\end{tikzcd}
$$

ℓ	gen_{LV}	$kill_{LV}$
1	$\emptyset$	$\{a\}$
2	$\{a\}$	$\{b\}$
3	$\{b\}$	$\{c\}$
4	$\{b\}$	$\{a\}$
5	$\{a\}$	$\emptyset$

ℓ	$LV_{entry}(\ell)$	$LV_{exit}(\ell)$
1	$LV_{exit}(1) \setminus \{a\}$	$LV_{entry}(2)$
2	$LV_{exit}(2) \setminus \{b\} \cup \{a\}$	$LV_{entry}(3)$
3	$LV_{exit}(3) \setminus \{c\} \cup \{b\}$	$LV_{entry}(4)$
4		

Example for LVA

$$
\begin{tikzcd}
{[a := 0]^1} \arrow[d] & \\
{[b := a * 1]^2} \arrow[d] & \\
{[c := c + b]^3} \arrow[d] & \\
{[a := b * 2]^4} \arrow[d] & \\
{[a < 9]^5} \arrow[uuu, bend left=60] \arrow[r] & {[\textbf{return } c]^6}
\end{tikzcd}
$$

ℓ	gen_{LV}	kill_{LV}
1	$\emptyset$	$\{a\}$
2	$\{a\}$	$\{b\}$
3	$\{b\}$	$\{c\}$
4	$\{b\}$	$\{a\}$
5	$\{a\}$	$\emptyset$

ℓ	$LV_{\text{entry}}(\ell)$	$LV_{\text{exit}}(\ell)$
1	$LV_{\text{exit}}(1) \setminus \{a\}$	$LV_{\text{entry}}(2)$
2	$LV_{\text{exit}}(2) \setminus \{b\} \cup \{a\}$	$LV_{\text{entry}}(3)$
3	$LV_{\text{exit}}(3) \setminus \{c\} \cup \{b\}$	$LV_{\text{entry}}(4)$
4	$LV_{\text{exit}}(4) \setminus \{a\} \cup \{b\}$	

Example for LVA

$[a := 0]^1$

$\downarrow$

$[b := a * 1]^2$

$\downarrow$

$[c := c + b]^3$

$\downarrow$

$[a := b * 2]^4$

$\downarrow$

$[a < 9]^5 \longrightarrow [\textbf{return } c]^6$

ℓ	gen_{LV}	$kill_{LV}$
1	$\emptyset$	$\{a\}$
2	$\{a\}$	$\{b\}$
3	$\{b\}$	$\{c\}$
4	$\{b\}$	$\{a\}$
5	$\{a\}$	$\emptyset$

ℓ	$LV_{entry}(\ell)$	$LV_{exit}(\ell)$
1	$LV_{exit}(1) \setminus \{a\}$	$LV_{entry}(2)$
2	$LV_{exit}(2) \setminus \{b\} \cup \{a\}$	$LV_{entry}(3)$
3	$LV_{exit}(3) \setminus \{c\} \cup \{b\}$	$LV_{entry}(4)$
4	$LV_{exit}(4) \setminus \{a\} \cup \{b\}$	$LV_{entry}(5)$
5		

Example for LVA

$$\begin{tikzcd}
{[a := 0]^1} \arrow[d] \\
{[b := a * 1]^2} \arrow[d] \\
{[c := c + b]^3} \arrow[d] \\
{[a := b * 2]^4} \arrow[d] \\
{[a < 9]^5} \arrow[r] \arrow[uuu, bend left] & {[\textbf{return } c]^6}
\end{tikzcd}$$

ℓ	gen_{LV}	kill_{LV}
1	$\emptyset$	$\{a\}$
2	$\{a\}$	$\{b\}$
3	$\{b\}$	$\{c\}$
4	$\{b\}$	$\{a\}$
5	$\{a\}$	$\emptyset$

ℓ	$LV_{\text{entry}}(\ell)$	$LV_{\text{exit}}(\ell)$
1	$LV_{\text{exit}}(1) \setminus \{a\}$	$LV_{\text{entry}}(2)$
2	$LV_{\text{exit}}(2) \setminus \{b\} \cup \{a\}$	$LV_{\text{entry}}(3)$
3	$LV_{\text{exit}}(3) \setminus \{c\} \cup \{b\}$	$LV_{\text{entry}}(4)$
4	$LV_{\text{exit}}(4) \setminus \{a\} \cup \{b\}$	$LV_{\text{entry}}(5)$
5	$LV_{\text{exit}}(5) \cup \{a\}$	

Example for LVA

$$
\begin{tikzcd}
{[a := 0]^1} \arrow[d] & \\
{[b := a * 1]^2} \arrow[d] & \\
{[c := c + b]^3} \arrow[d] & \\
{[a := b * 2]^4} \arrow[d] & \\
{[a < 9]^5} \arrow[r] \arrow[uuu, bend left=60] & {[\textbf{return } c]^6}
\end{tikzcd}
$$

ℓ	gen_{LV}	$kill_{LV}$
1	$\emptyset$	$\{a\}$
2	$\{a\}$	$\{b\}$
3	$\{b\}$	$\{c\}$
4	$\{b\}$	$\{a\}$
5	$\{a\}$	$\emptyset$

ℓ	$LV_{entry}(\ell)$	$LV_{exit}(\ell)$
1	$LV_{exit}(1) \setminus \{a\}$	$LV_{entry}(2)$
2	$LV_{exit}(2) \setminus \{b\} \cup \{a\}$	$LV_{entry}(3)$
3	$LV_{exit}(3) \setminus \{c\} \cup \{b\}$	$LV_{entry}(4)$
4	$LV_{exit}(4) \setminus \{a\} \cup \{b\}$	$LV_{entry}(5)$
5	$LV_{exit}(5) \cup \{a\}$	$LV_{entry}(2) \cup LV_{entry}(6)$

Results

$$[a := 0]^1$$
$$\downarrow$$
$$[b := a * 1]^2$$
$$\downarrow$$
$$[c := c + b]^3$$
$$\downarrow$$
$$[a := b * 2]^4$$
$$\downarrow$$
$$[a < 9]^5 \longrightarrow [\textbf{return } c]^6$$

ℓ	gen_{LV}	kill_{LV}
1	$\emptyset$	$\{a\}$
2	$\{a\}$	$\{b\}$
3	$\{b\}$	$\{c\}$
4	$\{b\}$	$\{a\}$
5	$\{a\}$	$\emptyset$

ℓ	$LV_{\text{entry}}(\ell)$	$LV_{\text{exit}}(\ell)$
1	$\emptyset$	$\{a\}$
2	$\{a\}$	$\{b\}$
3	$\{b\}$	$\{b, c\}$
4	$\{b, c\}$	$\{a, c\}$
5	$\{a, c\}$	$\{a, c\}$

Results

$$
\begin{tikzcd}
{[a := 0]^1} \arrow[d] & \\
{[b := a * 1]^2} \arrow[d] & \\
{[c := c + b]^3} \arrow[d] & \\
{[a := b * 2]^4} \arrow[d] & \\
{[a < 9]^5} \arrow[uuu, bend left=60] \arrow[r] & {[\textbf{return } c]^6}
\end{tikzcd}
$$

ℓ	gen_{LV}	kill_{LV}
1	$\emptyset$	$\{a\}$
2	$\{a\}$	$\{b\}$
3	$\{b\}$	$\{c\}$
4	$\{b\}$	$\{a\}$
5	$\{a\}$	$\emptyset$

ℓ	$LV_{\text{entry}}(\ell)$	$LV_{\text{exit}}(\ell)$
1	$\emptyset$	$\{a\}$
2	$\{a\}$	$\{b\}$
3	$\{b\}$	$\{b, c\}$
4	$\{b, c\}$	$\{a, c\}$
5	$\{a, c\}$	$\{a, c\}$

Note: b and a are never simultaneously live!

Existence of Solutions

Solutions **always exist** to our data flow equations.
Why? Because $(2^{\mathcal{A}}, \subseteq)$ (for AEA) and $(2^{\mathsf{Var}}, \subseteq)$ (for LVA) are both lattices and all our functions are monotone. So the Knaster-Tarski theorem applies.

Bibliography

- F. Neilson: Principles of Program Analysis, Chapters 2 and 4, Springer 1999
- P. Cousot, A Tutorial on Abstract Interpretation, VMCAI 2005.
- Aho, Lam, Sethi, Ullman: Compilers: Principles Techniques and Tools (the Dragon Book), Second Edition.
- 2022 Vistas in Verification talk by Moogli Sagiv